

How SKYNET Works

The technical architecture, explained for the team.

What runs where, why it's split that way, and what happens to a single social media post between the internet and the screen.

Problem statement	SIH26152 — Social Media Analytics · Sponsor NTRO
Team	SKYNET
Repo	Krishna-Das20/SIH-26152
Styled web version	https://claude.ai/code/artifact/c1ad3f65-f20c-4cb6-acd2-953c991bc27c

Every figure in this document was measured on the running system, not estimated. If a number isn't here, don't put it on a slide.

1. What the system does

Give it a YouTube video, an Instagram reel, a Telegram channel or a subreddit. It collects the posts and comments, works out *what people feel* and *who is saying it*, finds *which topics are rising*, maps *how those people are connected* — and then, crucially, **combines all four** into findings none of them could produce alone.

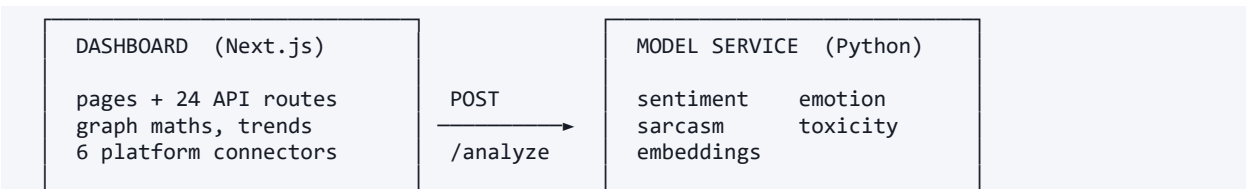
That last step is the whole point. The problem statement says so directly:

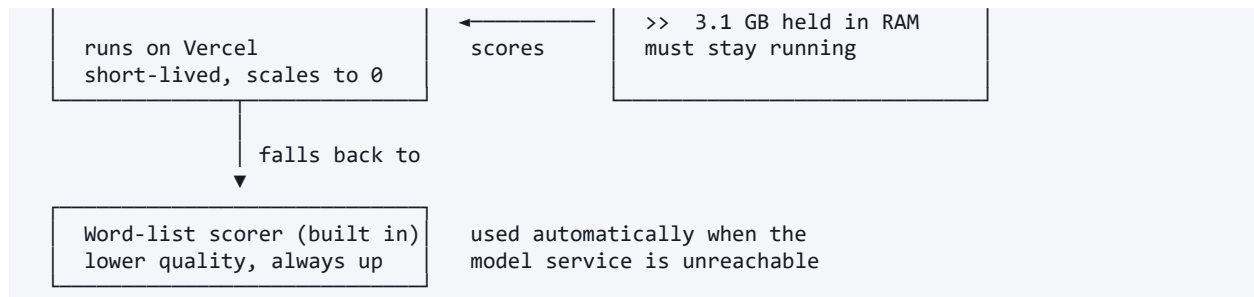
"Combining these four vectors using AI is the key to unlocking true audience intelligence."

Measure	Value
Lines of TypeScript	21,110
Lines of Python	3,416
API endpoints	24
Transformer models	5
Platform connectors	6

2. The one big decision: two processes

Everything else follows from this. The system is **two programs that talk over HTTP**, not one.





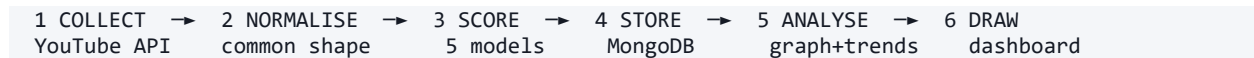
Why they can't be one program. The five models occupy 3.3 GB on disk and **3.1 GB in memory** once loaded. A serverless function is created, answers one request, then is destroyed — so it would reload 3.3 GB every single time, taking about **34 seconds**, against execution limits measured in seconds.

The models need a process that stays alive. Web traffic needs processes that don't. Those are opposite requirements, so they are opposite processes.

Say this to a judge: "We split it because the constraint is physical, not stylistic. Models need memory that persists; web traffic needs processes that don't. One environment can't do both well, so we let each side do what it's good at and joined them with one HTTP call."

3. The journey of one post

This is the clearest way to explain the system. Follow a single YouTube comment from the internet to a pixel on screen.



1 • Collect. The connector calls the platform's real API and gets raw JSON.

2 • Normalise. It's rewritten into one common shape — a **SocialPost** with author, text, timestamp, likes, replies and a link — so the rest of the system never needs to know which platform it came from.

3 • Score. Sent in batches of 12 to the model service. Comes back with sentiment, emotion, a sarcasm score, an inferred stance, and a list of **384 numbers** representing its meaning.

4 • Store. Saved to MongoDB with its scores, de-duplicated by post id.

5 • Analyse. Who-replies-to-whom becomes a graph. Word counts over time become trends. Posts with similar 384-number vectors become narrative clusters.

6 • Draw. The dashboard reads the finished numbers and renders them.

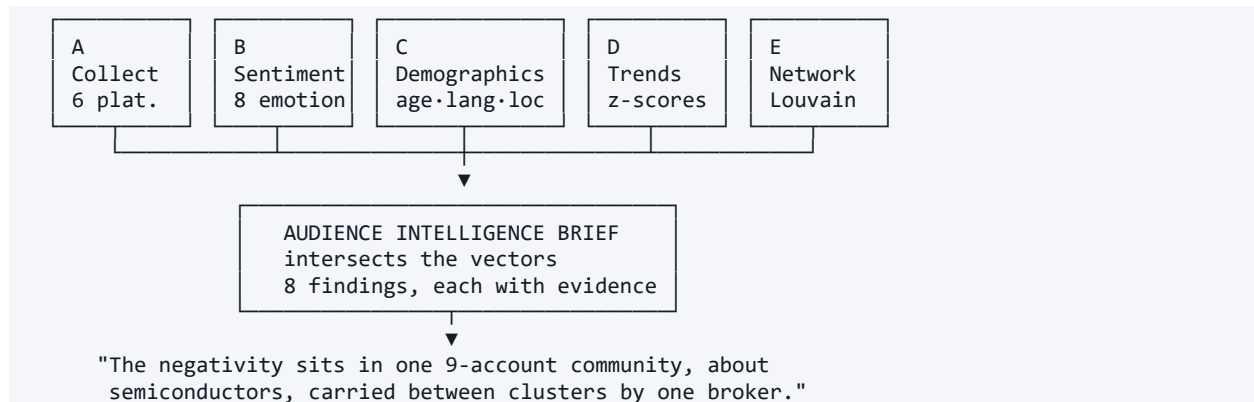
Steps 1–4 happen when you ingest. Step 5 runs on every page load, over whatever is in the store. **A judge pasting a YouTube link sees a fully re-scored dashboard in about 12 seconds.**

What "384 numbers" means

An **embedding** turns a sentence into a list of 384 numbers, positioned so that sentences meaning similar things end up close together. That is how the system groups posts into narratives without anyone writing rules about topics — it measures distance between *meanings*, not matching keywords.

4. The five components, and the layer above them

The problem statement asks for five things. Most teams will build five panels. The sixth box is the one that matters.



Why the sixth box is the project. Any dashboard can say "62% negative". Only a layer that holds sentiment *and* network topology **at the same time** can say which community the negativity lives in, what it is about, and who is carrying it between groups.

Each finding ships with the evidence it was derived from, so a reader can check it rather than trust it.

5. The five models

These are published, peer-reviewed models downloaded from Hugging Face — not something trained on a laptop. Each does one job.

Job	Model	Size
Positive / negative / neutral	cardiffnlp/twitter-roberta-base-sentiment-latest	957 MB
Emotion (joy, anger, fear...)	SamLowe/roberta-base-go_emotions	479 MB
Sarcasm detection	hallisky/sarcasm-classifier-gpt4-data	1.4 GB
Toxicity	unitary/toxic-bert	418 MB
Meaning vectors	sentence-transformers/all-MiniLM-L6-v2	88 MB

The sentiment model was trained on **tweets**, not books — which matters, because social media language is short, sarcastic and full of emoji. The sarcasm model was chosen by testing four candidates on real examples; this one scored 0.999 on clear sarcasm with zero false positives on the control set.

Expect this question: "Did you train these yourself?" No, and that's the right call. Fine-tuning a sentiment model on a few hundred posts would perform *worse* than a model trained on millions. What we engineered

is the layer above: choosing models empirically, mapping their outputs onto the five dimensions the problem statement names, and fusing them.

6. The algorithms we implemented ourselves

The network analysis is not a library call. Three classic algorithms, written out by hand in `src/lib/graph/`:

Algorithm	Answers	Paper
Louvain	Which accounts form a community?	Blondel et al., 2008
Brandes betweenness	Who is the bridge between communities?	Brandes, 2001
PageRank	Who is genuinely influential, not just loud?	Page & Brin, 1998

Betweenness is the interesting one. It finds accounts that sit on the path between groups that otherwise don't talk to each other. An account with few followers can still be the only bridge between two communities — which makes it far more important to an analyst than a popular account talking to people who already agree with it.

Be ready for this. Our graph modularity is **0.2796**, below the 0.3 convention for "real community structure". That is a true property of this corpus, not a broken algorithm: nearly half our posts are Instagram comments replying to one account, which is a star shape with no communities to find. On discussion-heavy sources the same code measures 0.83. Saying this *before* a judge asks is worth more than the number would be.

7. The rule that makes it defensible

The system never invents a number. When a platform doesn't report something, the field is `null` and the screen says "Unknown" or "n/a".

- Reddit's public feed carries no like count, so `likes` is `null` and shows as "n/a" — **not 0**, because zero is a claim.
- Age can't be inferred for 92% of authors, so the demographics page says **92% Unknown** on screen.
- If the model service is down, each post records which engine scored it, so the interface can state whether you are looking at transformer output or fallback.

Earlier versions of this codebase had seven places generating `Math.random()` values to fill gaps. All were removed, and a test suite now **fails the build** if fabricated engagement reappears.

Why this wins points. Half the projects in the room will show plausible-looking numbers that came from nowhere. A judge who probes one and finds it invented will discount everything else that team said. Ours survives the probe — and volunteering our weak spots first is what makes the strong claims believable.

8. Where the code lives

Folder	What's in it
<code>src/lib/ingestion/</code>	The six platform connectors, one file each, behind a shared interface
<code>src/lib/ml/</code>	The bridge to the Python service, plus the word-list fallback
<code>src/lib/graph/</code>	Louvain, Brandes, PageRank
<code>src/lib/narratives/</code>	Clustering posts by meaning, scoring how a story mutates
<code>src/lib/nlp/</code>	Rule-based demographics and the fallback scorer
<code>src/app/api/</code>	24 endpoints the dashboard calls
<code>ml/</code>	The Python service — one folder per model

Two design choices worth naming

Fault isolation. The six connectors run in parallel and are wrapped individually. If Instagram's token expires mid-demo, the other five still return data and the interface reports exactly why Instagram is empty. One platform failing never takes the page down.

Graceful degradation. Every page except Narratives works with the Python service switched off, because the demo corpus ships pre-scored. You can prove this on stage: stop the model service, reload, and watch everything keep working while the one dependent panel names its own cause.

9. Numbers you can quote

Measure	Value	Note
Posts in the demo corpus	358	real captures, frozen so the demo needs no wi-fi
Transformer-scored	358 / 358	100% — no fallback scores
Accounts in the graph	298	174 real reply / mention edges
Cross-vector findings	8	each carries its evidence
Model throughput	9.3 / sec	on a laptop CPU, no GPU
Cold start	~34 s	14 s to load + 20 s first batch
Live ingest, end to end	~12 s	paste a link → re-scored dashboard

10. The single best live demo moment

Stop the Python service on stage and reload the page.

Everything keeps working except Narratives, which names its own cause. That one action demonstrates the two-process split, graceful degradation and honest error reporting in about fifteen seconds — better than any slide about it.

Section 8 explains why it works, if someone asks.

Compiled 28 Aug 2026. All figures measured on the running system. Related: [PROGRESS.md](../PROGRESS.md) · team-brief.md · [AGENTS.md](../AGENTS.md)